

Natural Language Processing

[DSAI 3308]



Lecture 02: Text Processing

Data Collection

- Gather text data from various sources such as websites, books, articles and social media.

➤ Challenges:

➤ Data Quality

- Incomplete or missing data
- Inconsistent data formats
- Presence of noise and outliers

➤ Data Privacy and Security

- Ensuring data anonymization
- Compliance with regulations
- Securing data storage and transfer

Data Collection

➤ Data Accessibility

- Limited access to proprietary or sensitive data
- High costs of acquiring certain dataset
- Technical barriers to accessing data from various sources

➤ Data volume and variety

- Managing large volume of data (Big Data)
- Integrating data from multiple sources and formats
- Handling unstructured data (e.g. text, images, videos)

➤ Bias and representativeness

- Ensuring the data is representative of the population
- Avoiding sampling bias
- Addressing any inherent biases in the data collection process

Text Cleaning

➤ Remove Noise:

- Punctuation, Numbers, and Special characters:
 - Original: Hello! This is an example text with numbers 123456 and symbols \$#@%.
 - Cleaned: Hello This is an example text with numbers and symbols
- Removing noise helps focus on meaningful parts of the text.

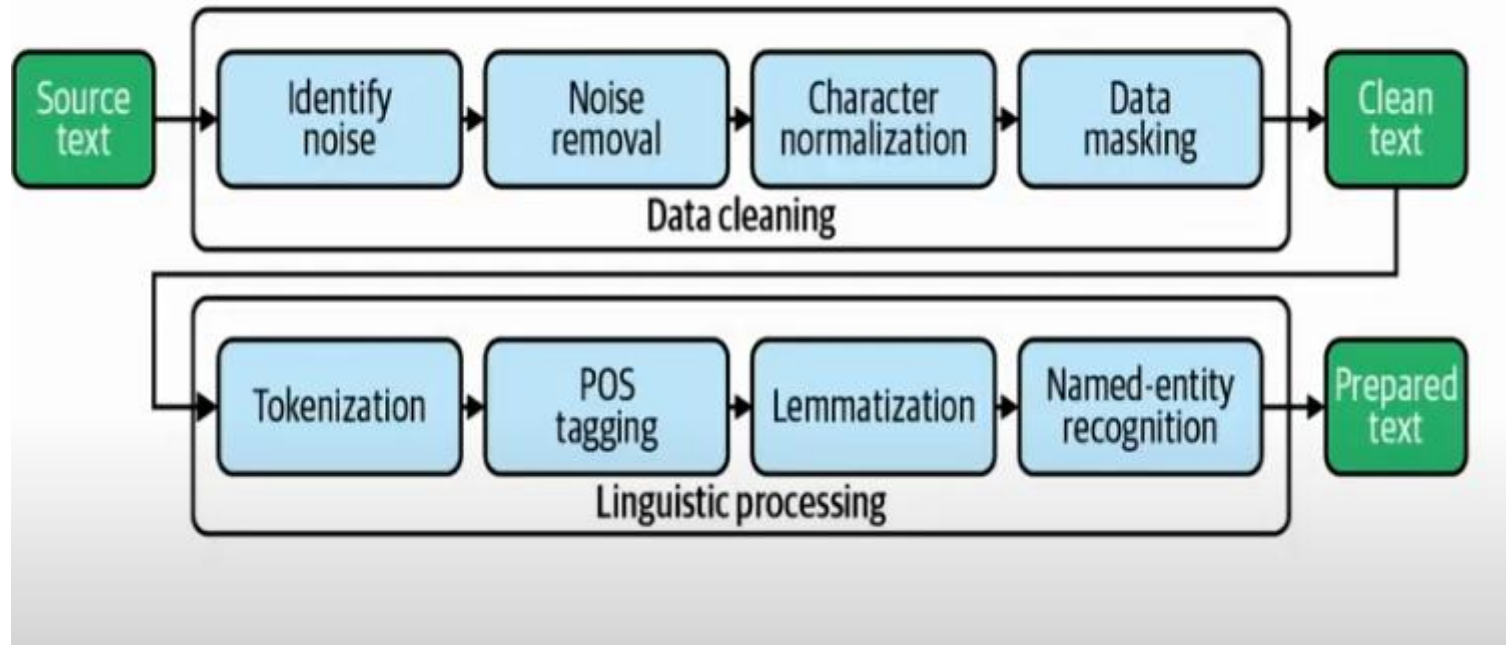
➤ Correct Spelling Errors and Normalize Text:

- Original: “ This sentnce contains a speling error.”
- Corrected: “ This sentence contains a spelling error.”
- Normalization involves converting text to a standard form, such as converting different forms of a word to a single form. (e.g. “color” and “colour” to “color”)

➤ Handle Misspellings, Slang, and Abbreviations:

- Original: “ OMG, this txt is gr8!”
- Normalized: “ Oh my god, this text is great!”
- Converting slang and abbreviations to their full forms ensures clarity and consistency

Text Preprocessing



Text preprocessing is crucial for improving the quality of text data before applying NLP techniques.

Text Preprocessing

- It improves the quality of text data before applying NLP techniques.
- **Enhances Accuracy:** Clean and well-processed text improves the performance of NLP tasks like parsing and named entity recognition.
- **Reduce Noise:** Removing irrelevant information (e.g. stop words) helps focus in meaningful content.
- **Facilitates consistency:** Normalization techniques ensure uniformity in text data, aiding better understanding and analysis.
- **Improve training efficiency:** Preprocessed text speeds up training by reducing complexity and dimensionality.
- **Boosts Model Quality:** Clean and standardized data helps in learning more accurate language patterns.
- **Mitigates Bias:** Proper preprocessing can help in reducing biases present in the raw text.

Text Preprocessing

Text Preprocessing Steps

- **Tokenization:** Split Text into individual words or sentences.
- **Lowercasing:** convert all text to lowercase to ensure consistency.
- **Stop words removal:** Eliminate common words (e.g. “and”, “the”) that add little value.
- **Normalization:** convert text into a standardized format by addressing various inconsistencies and variations.
- **Stemming/Lemmatization:** Reduce words to their base or root form.

Text Preprocessing

- Tokenization is the process of splitting text into smaller units called tokens (sentences or words).
- It is a technique that involves dividing a phrase, sentence, paragraph or an entire text document into smaller units known as tokens.
- It helps machine understand and process human language by breaking it into manageable pieces.
- It helps in interpret the meaning of the text by analyzing the words present in the text.
- Count the number of the words in the text

Text Preprocessing

- **Tokenization Use Cases**

1. Search engines.
2. Machine translation.
3. Speech recognition.
4. Sentiment analysis in reviews.
5. Chatbots and virtual assistants.

- **Tokenization Challenges**

- Ambiguity.
- Languages without clear boundaries.
- Handling special characters.

Text Preprocessing

Tokenization

```
graph TD; A[Tokenization] --> B[Sentence]; A --> C[Word];
```

Sentence

is the process of splitting text into individual sentences.

Word

is the process of splitting text into individual words.

Text Preprocessing

Sentence tokenization

- **Original text:** “ Dr.Ahmed, an expert in AI, visited Gaza. He gave a talk on Ph.D. research as UCAS. His presentation was insightful, e.g., he discussed various algorithms. After the event, we went to ‘Garden Beach’ for a relaxing evening.”
- **Sentenced Tokenized text:**
 - “Dr.Ahmed, an expert in AI, visited Gaza.”
 - “He gave a talk on Ph.D. research as UCAS.”
 - “His presentation was insightful, e.g., he discussed various algorithms.”
 - “After the event, we went to ‘Garden Beach’ for a relaxing evening.”

Text Preprocessing

Word tokenization

“NLP helps machines understand language.”



NLP

helps

machines

understand

language.

Implementing Tokenization

- **NLTK (Natural Language Toolkit):** is a comprehensive Python library that caters to a wide range of linguistic needs. It offers both word and sentence tokenization functionalities, making it a versatile choice for beginners and seasoned practitioners alike.
- **Spacy:** A modern and efficient alternative to NLTK, Spacy is another Python-based NLP library. It boasts speed and supports multiple languages, making it a favorite for large-scale applications.
- **BERT tokenizer:** Emerging from the BERT pre-trained model, this tokenizer excels in context-aware tokenization. It's adept at handling the nuances and ambiguities of language, making it a top choice for advanced NLP projects

Lowercasing

- **Lowercasing** is the process of converting all characters in a text to lowercase. This step standardizes text data by eliminating case differences, which helps in uniform analysis.
- Why lowercasing is important?
 - ❑ **Uniform representation:** Treats words with different cases as identical, which is crucial for accurate text analysis and preprocessing.
 - ❑ **Simplifies matching:** helps in text matching and retrieval tasks by reducing case sensitivity.
 - ❑ **Improves model efficiency:** Ensures that the text data is consistent, enhancing the performance of machine learning models.

Lowercasing

❖ Original Text:

“The quick brown Fox jumps over the lazy DOG.”

❖ After lowercasing:

“the quick brown fox jumps over the lazy dog.”

Use cases:

- Consider a search engine querying for “quick Brown fox” in a database of documents.
- Lowercasing ensures that the search results match regardless of case used in the query of the documents.